Project Overview

Our database application uses mysql and python to manage a game cafe's operations. The app's entry point is ui.py. From this terminal UI, the can start new game sessions, modify tables, modify views, and conduct various reports/analytics (queries). The UI prompts the user as follows:

```
1. Start a new Session
2. EMPLOYEE TABLE
3. MENU ITEMS TABLE
4. CUSTOMERS TABLE
5. VIEWS
6. REPORTS / ANALYTICS (Queries)
0. Exit
```

From here, each option contains a submenu for the specific action they'd like to perform.

We create the mysql database server using schema.sql and data.sql. 'frontend/ui.py' manages the terminal interface and action selection. This includes printing the options to console, receiving input, and calling the correct logic from that input. The database logic resides in backend/session_logic.py. Here, we execute every definition, manipulation, and query statement safely using the mysql-connector-python package.

Schema Summary

Our schema manages a game cafe business using the following tables and relationships:

Core Entities:

- Employee: Manages store staff and their information (name, role, wage)
- Menu_Item: Manages the menu items (name, category, price)
- Customer: Managers the customers name, email, and membership
- Station: Manages the gaming station type, rate, and status
- Game: Manages game the title, genre, and platform

Relationship Entities:

- Station_Game: A junction table between Station and Game, creating a many to many relationship. It indicates which games are installed on each station.
- Session: One to many relationship. Links customers and a session for a specific time window. Tracks their session time at a Station.

- Order: Creates a one to many relationship. Links Customer and Employee. Also records the order time.
- Order_Item: Many to many relationship (junction). Links Orders and Menu_Item. A single order can contain multiple menu items.
- Order_Status: One to many relationship with Orders. Stores the status, updated_at, and notes for an order lifecycle.

Advanced Features

- How they are integrated into the application
    - Created two views(view_session_summary, view_station_availability) both of which help group together important, correlated information in one place. Integrated into the application with a menu option, view_station_availability gets information about the Station, Station_game, and Game tables, allowing them to know what stations have what games, and if it is open. view_session_summary pulls from Session, Customer, and Station tables, allowing the user to see who used a station and for how long.
    - (this was alec - TRIGGER statement)

Individual Contribution Statement

1. **Michael Venous:**

Tables created: Session Table
Queries written: Queries 1-4 -

```
-- Query 1: All of the information from the Customer table
SELECT * FROM Customer;

-- Query 2: All sessions where there is no end time
SELECT * FROM Session WHERE end_time IS NULL;

-- Query 3: The name of the Customer and the total amount of sessions that they
have
SELECT c.Name, COUNT(s.session_id) AS total_sessions
FROM Customer c JOIN Session s ON c.Customer_id = s.customer_id
GROUP BY c.Name;

-- Query 4: All games that are going to be available at each station with station
type
SELECT s.station_id, s.station_type, g.title, g.genre, g.difficulty
FROM Station s
JOIN Station_Game sg ON s.station_id = sg.station_id
```

JOIN Game g ON sg.game_id = g.game_id;

Back-end:
Added the python logic to execute the general queries 1-9
From the UI, created the option “6. REPORTS / ANALYTICS (Queries)” which allows you to call queries 1-9.

Front-end: Created front end UI with the help of gemini. The front end calls the back-end function which safely executes the corresponding sql statement.

2. Nikolas Enriquez:

Tables created

- Station
- Game
- Station_game

Advanced Features

- Created View view_session_summary

```
CREATE VIEW view_session_summary AS
SELECT
    se.session_id,
    c.Name         AS customer_name,
    c.email,
    st.station_type,
    se.start_time,
    se.end_time,
    se.total_cost,
    TIMESTAMPDIFF(MINUTE, se.start_time, se.end_time) AS duration_minutes
FROM Session se
JOIN Customer c  ON se.customer_id = c.Customer_id
JOIN station st  ON se.station_id  = st.station_id;
```

- Created view, view_station_availability

```
CREATE VIEW view_station_availability AS
SELECT
    s.station_id,
    s.station_type,
    s.hourly_rate,
    s.availability,
    COUNT(g.game_id) AS num_games
FROM station s
LEFT JOIN station_game sg ON s.station_id = sg.station_id
LEFT JOIN game g ON sg.game_id = g.game_id
GROUP BY s.station_id, s.station_type, s.hourly_rate, s.availability;
```

Queries written

- One to view important information on view_session_summary

```
cur.execute("""
    SELECT session_id, customer_name, email, station_type,
           start_time, end_time, total_cost, duration_minutes
    FROM view_session_summary
    ORDER BY session_id
""")
```

- One to view general information on view_station_availability

```
cur.execute("""
    SELECT station_id, station_type, hourly_rate,
           availability, num_games
    FROM view_station_availability
    ORDER BY station_id
""")
```

Backend

- Created functions and logic for how data will get pulled and displayed when users interact with the database, creating code in session_logic.py

3. Aadil Qureshi:

- Tables worked on
  - Employee
  - MenuItem
  - Customer
- Queries written

```
SELECT * FROM Customer;
```

```
SELECT * FROM Session WHERE end_time IS NULL;
```

```
SELECT c.Name, COUNT(s.session_id) AS total_sessions
FROM Customer c JOIN Session s ON c.Customer_id = s.customer_id
GROUP BY c.Name;
```

- Backend features implemented
  - run_queries.py which looks at the queries written in queries.sql and writes the results over to query_output.txt

AI Use Disclosure

For the front end (frontend/ui.py), we used Gemini 3.1 Pro to assist with the repetitive menu prompts and if else statements.